



Chapter 1: Introduction

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- **The Need for Databases** 数据库的重要性
- **Data Models** 数据模型
- **Relational Databases** 关系数据库
- **Database Design** 数据库设计 必考试难的 ——— 数据库引擎
- **Storage Manager** 存储管理器 仅第一章讲
- **Query Processing** 查询处理 后面不讲 但要了解
- **Transaction Manager** 事务管理器 专门有东西讲



Database Management System (DBMS)

DBMS包含特定企业的信息

什么是DBMS?

- DBMS contains information about a particular enterprise

- Collection of interrelated data 相关联数据集合

数据库引擎

- Set of programs to access the data 一些能访问数据的程序

- An environment that is both *convenient* and *efficient* to use 一个方便且高效的使用环境



- Database Applications: 数据库应用

没
用

- Banking: transactions 银行: 交易

- Airlines: reservations, schedules 航空公司: 预定, 安排

- Universities: registration, grades 大学: 登记, 分数

- Sales: customers, products, purchases 销售: 顾客, 商品, 购买

- Online retailers: order tracking, customized recommendations

- Manufacturing: production, inventory, orders, supply chain

- Human resources: employee records, salaries, tax deductions

- Databases can be very large. 数据库可以很大

- Databases touch all aspects of our lives 数据库在我们生活的方方面面



数据库的基本概念

- 数据 (Data)
- 数据库 (Database, DB)
- 数据库管理系统 (Database Management System, DBMS)
- 数据库系统 (Database System, DBS)





University Database Example

- Application program examples 数据库应用举例
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts
- In the early days, database applications were built directly on top of file systems

早期, 数据库应用直接建立在文件系统上



Drawbacks of using file systems to store data

文件系统的缺点

- **Data redundancy and inconsistency** 数据冗余, 不一致
 - Multiple file formats, duplication of information in different files
- **Difficulty in accessing data** 难以访问
 - Need to write a new program to carry out each new task
- **Data isolation** 数据孤立
 - Multiple files and formats
- **Integrity problems** 完整性的问题
 - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones



Drawbacks of using file systems to store data (Cont.)

- **Atomicity of updates** 更新的原子性
 - Failures may leave database in an inconsistent state with partial updates carried out
 - Example: Transfer of funds from one account to another should either complete or not happen at all
- **Concurrent access by multiple users** 多个用户并发访问
 - Concurrent access needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - 4 Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- **Security problems** 安全性问题
 - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems



数据管理的三个阶段

— 20世纪50年代中期

20世纪50年代中期 — 60年代中期

20世纪60年代末 — 至今

人工管理

文件系统管理

数据库管理

	人工管理	文件系统管理	数据库管理
应用背景	满足科学计算	满足科学计算和数据管理	大规模数据管理
硬件条件	无直接存取存储设备	磁盘、磁鼓	大容量磁盘
软件条件	没有操作系统	有文件系统	数据库管理系统
处理方式	批处理	联机实时处理、批处理	联机实时处理、批处理、分布式处理
实现方式	通过卡带、卡片和磁带等计算机间接存取设备实现	通过磁盘和磁鼓等计算机直接存取设备实现	通过数据库管理系统与数据库进行交互以实现数据管理操作
特点	- 人力维护成本极高 - 数据与应用程序深度绑定 - 数据间缺乏组织和独立性 - 数据无共享、冗余度极大	- 人力维护成本很高 - 数据文件与应用程序耦合 - 数据共享性差、冗余度大 - 难以支持多用户并发访问	- 使用数据模型描述，无需应用程序定义，组织程度和共享程度高 - 编程友好，无需关心数据库文件的物理操作和系统控制 - 支持多用户并发访问



文件系统阶段

The Human Eye



劳资科

学号	姓名	系别	补贴
----	----	----	----

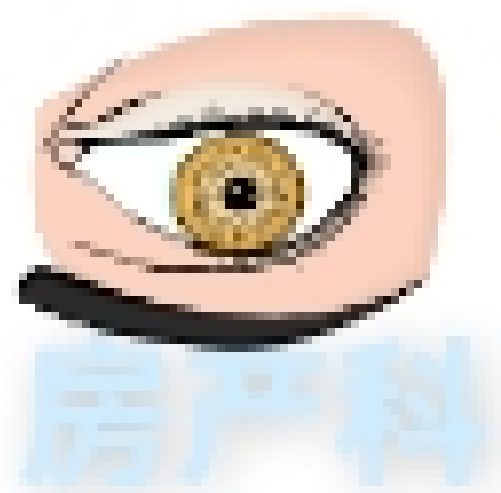
The Human Eye



学籍科

学号	姓名	系别	学分	学位
----	----	----	----	----

The Human Eye



房产科

学号	姓名	系别	性别	住址
----	----	----	----	----

The Human Eye

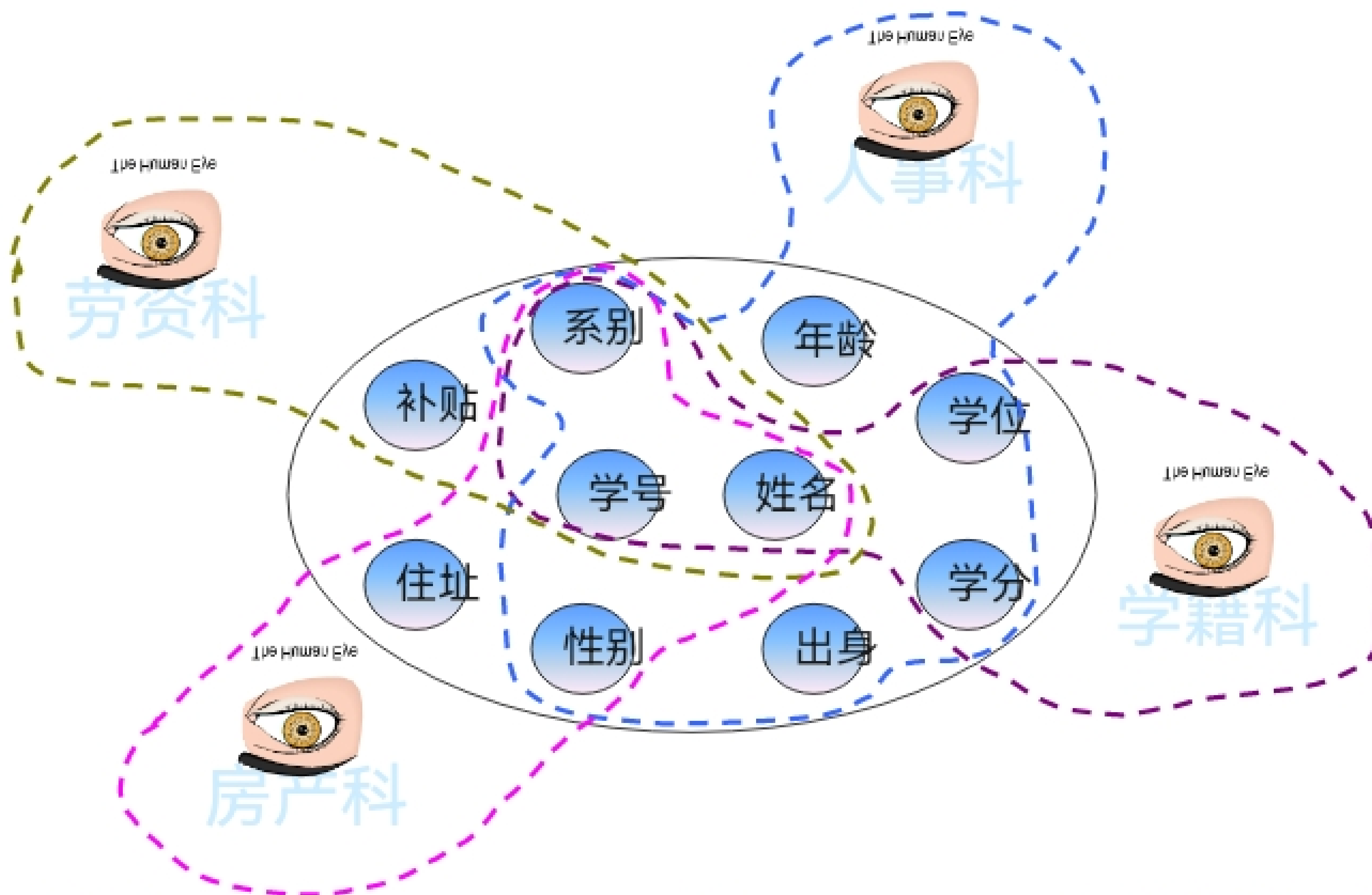


人事科

学号	姓名	系别	性别	学位	年龄	出身
----	----	----	----	----	----	----



数据库系统的特点





Levels of Abstraction 逻辑等级

决定数据库性能  物理层: 描述一个数据如何被存储进文件

- **Physical level:** describes how a record (e.g., instructor) is stored.

逻辑层: 描述数据及其相互关系存储在数据库中

- **Logical level:** describes data stored in database, and the relationships among the data.

type *instructor* = **record**

ID: string;

name: string;

dept_name: string;

salary: integer;

end;

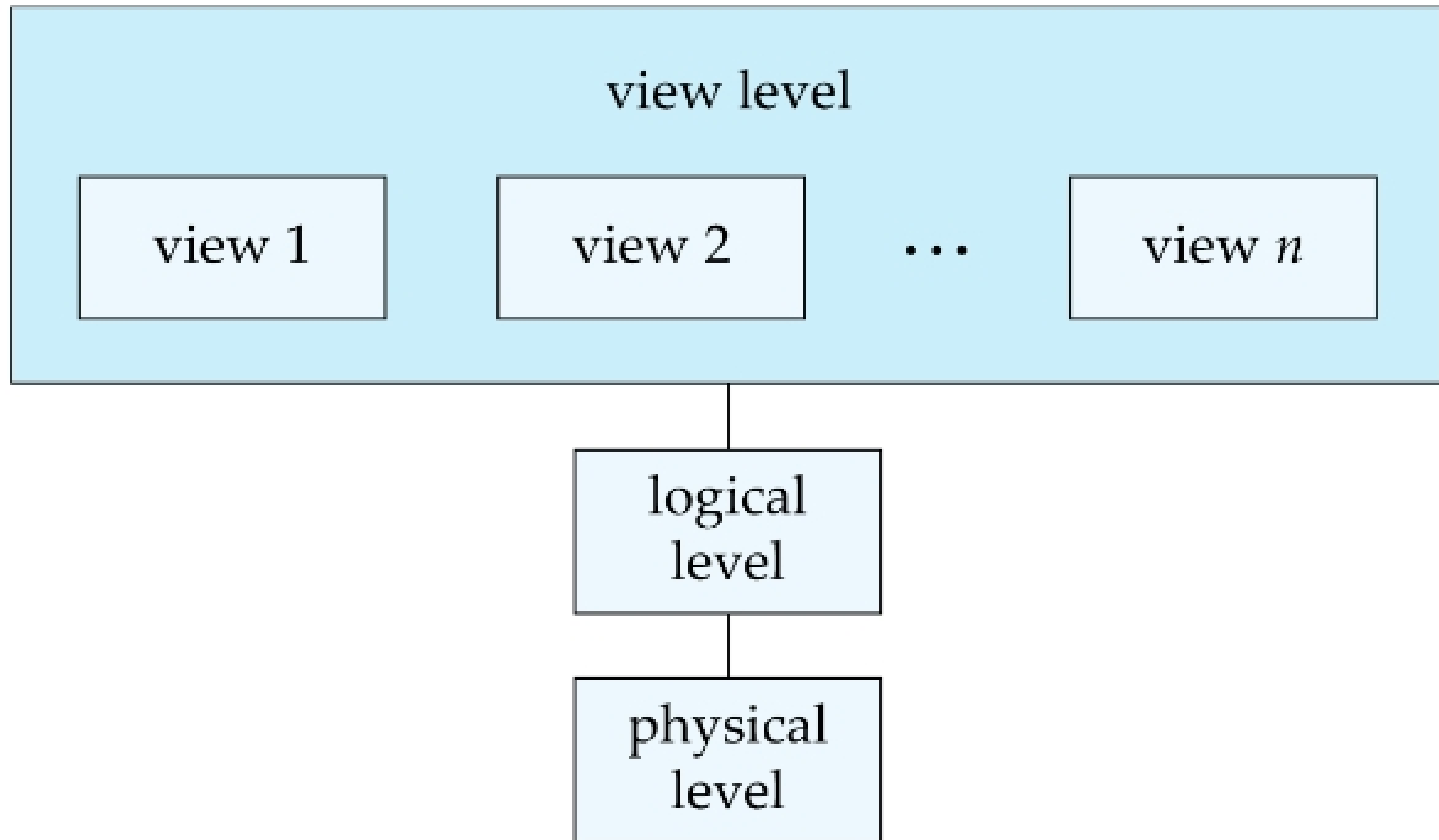
视图层

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.



View of Data

An architecture for a database system





Instances and Schemas

实例(当前的运行状态)

模式(逻辑结构)

数据库的模式和实例可类比编程语言写出的程序 模式——变量声明 实例=变量在某时刻的值

- Similar to types and variables in programming languages
- **Logical Schema** — the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
- **Physical schema** — the overall physical structure of the database
 - Analogous to type information of a variable in a program
- **Instance** — the actual content of the database at a particular point in time
 - Analogous to the value of a variable
- **Physical Data Independence** — the ability to modify the physical schema without changing the logical schema
 - Applications depend on the logical schema 应用程序仅依赖逻辑模式
 - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others. 好好设计很重要



Data Models

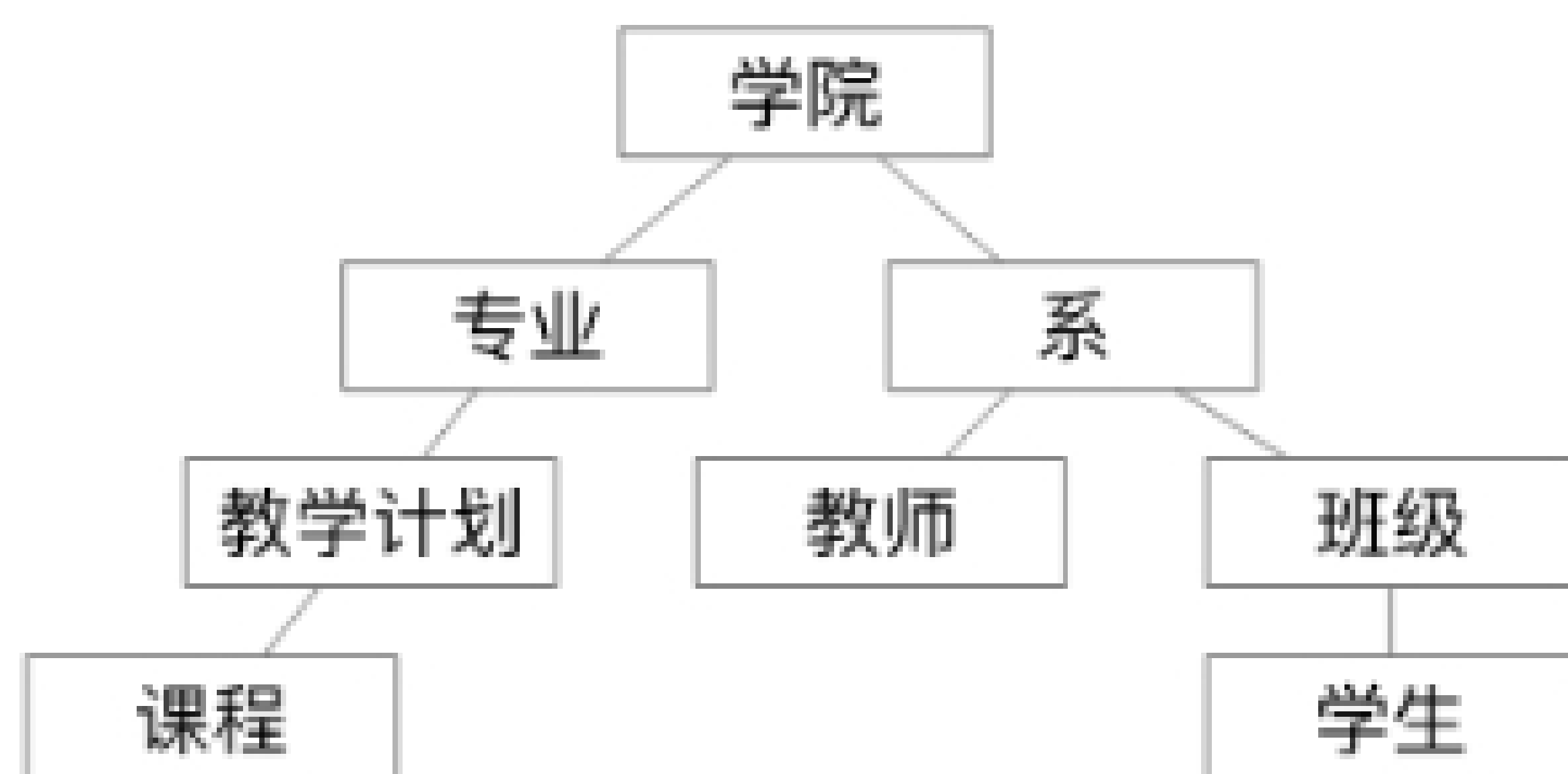
- A collection of tools for describing
 - Data
 - Data relationships 数据之间的联系
 - Data semantics 数据的语义信息
 - Data constraints 数据的约束—以保证数据的完整性(例分数100)
- Relational model 关系模型.
- Entity-Relationship data model (mainly for database design) 实体联系数据模型(考试必考)
- Object-based data models (Object-oriented and Object-relational) 对象关系数据模型
- Semistructured data model (XML) 半结构化数据模型
- Other older models:
 - Network model
 - Hierarchical model



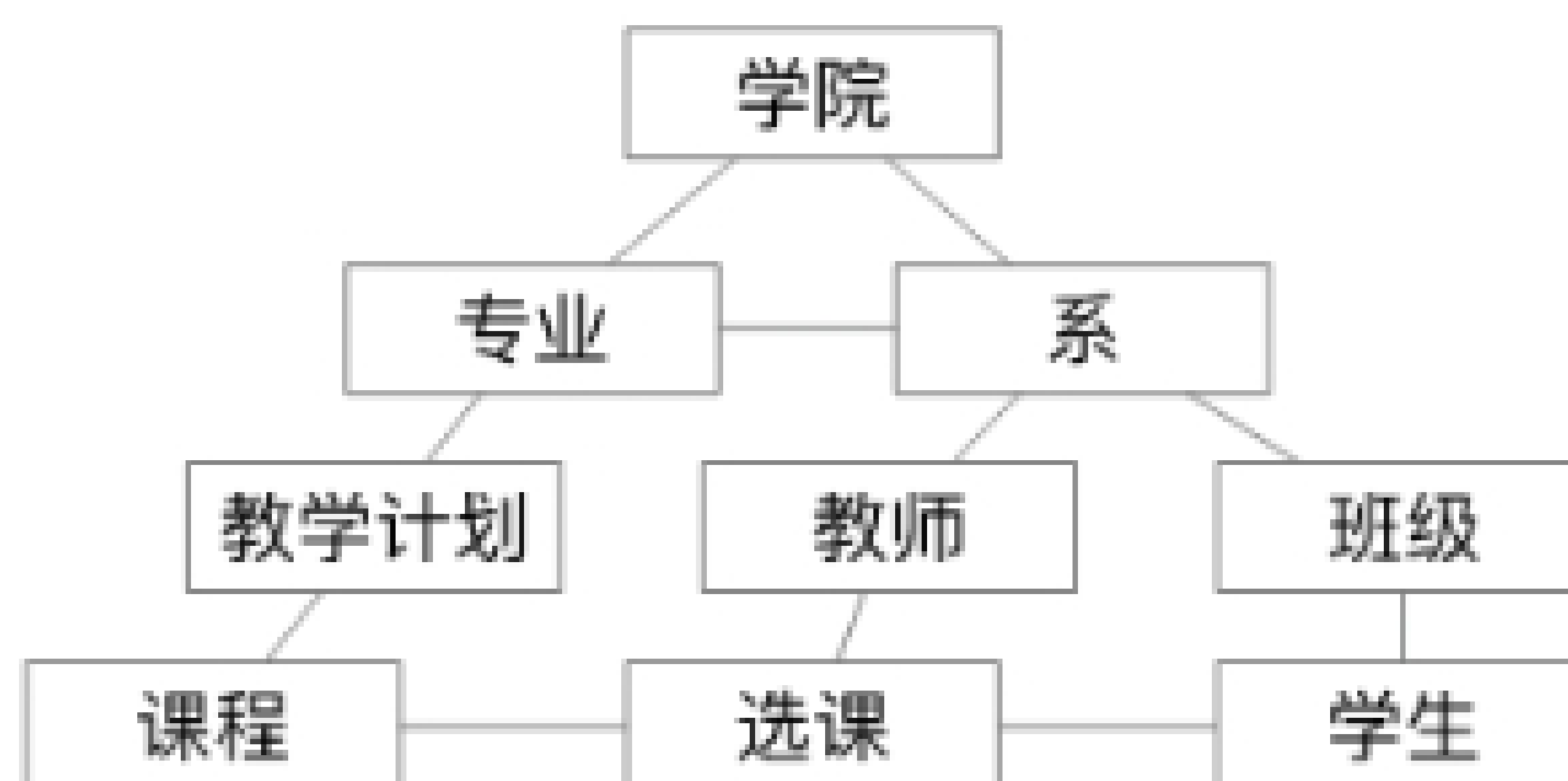
数据库模型：层次数据库和网状数据库

➤ 在数据库发展的早期，有两类重要的数据库

- 层次数据库 (Hierarchical Database) 和网状数据库 (Network Database)
- 层次数据库基于层次模型，以“树结构”来表示数据记录之间的联系
- 网状数据库基于网状模型，以“图结构”来表示数据记录之间的联系



(1) 层次数据模型

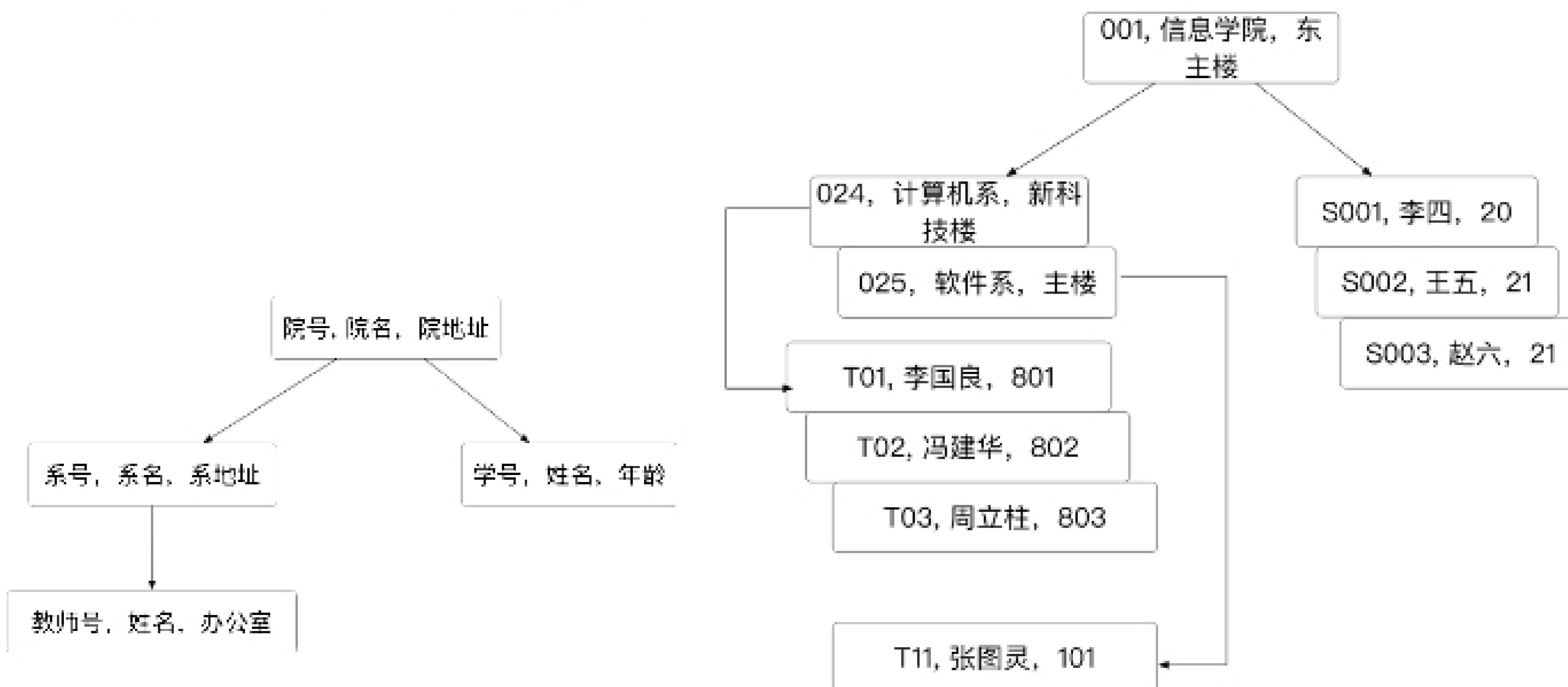


(2) 网状数据模型



数据库模型：层次模型与层次数据库

- 层次模型用树形结构来表示各类实体以及实体间的联系
 - 除根节点以外，其他结点有且只有一个父亲结点
- 典型代表是IBM公司的IMS（Information Management System, IMS）数据库

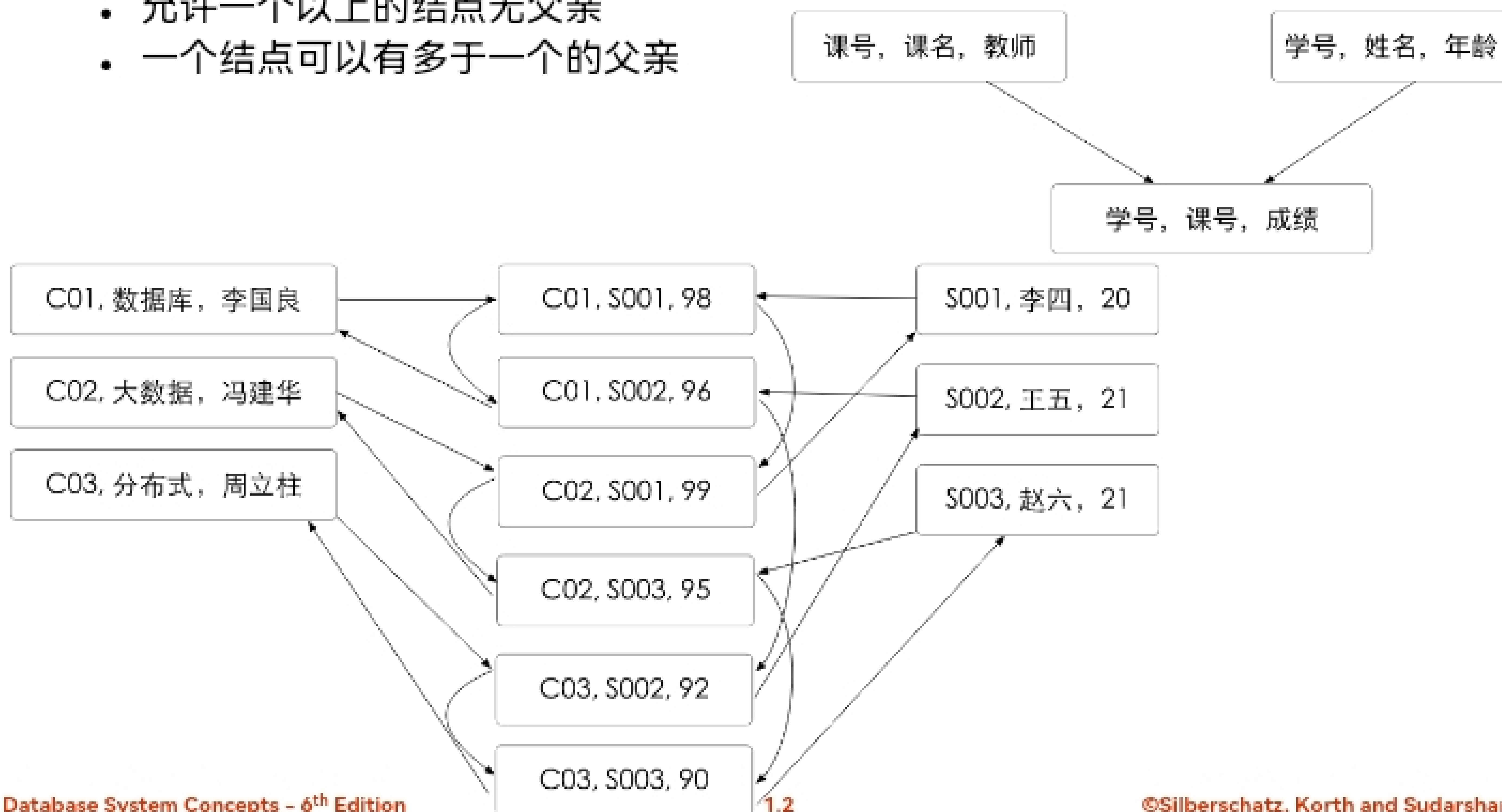




数据库模型：网状模型和网状数据库

➤ 美国通用电气公司发布了世界上第一个网状数据库系统——IDS (Integrated Data Storage, 集成数据存储)

- 允许一个以上的结点无父亲
- 一个结点可以有多于一个的父亲





Relational Model

关系模型用表的集合来存放数据

- All the data is stored in various tables.
- Example of tabular data in the relational model

Columns			
<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000
Rows			

(a) The *instructor* table



A Sample Relational Database

<i>id</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33156	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table



Data Definition Language (DDL)

用于定义数据库模式

数据定义语言

- Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a **data dictionary** ~~data~~ 数据字典, 简称DD, 存储元数据, 用户无权限写, 用来管理用户创建的表的元数据表
- Data dictionary contains metadata (i.e., data about data)
 - **Database schema** 数据概要
 - **Integrity constraints** 约束条件
 - 4 Primary key (ID uniquely identifies instructors)
 - **Authorization** 权限
 - 4 Who can access what



Data Manipulation Language (DML)

数据操纵语言(增删改查)

- Language for accessing and manipulating the data organized by the appropriate data model
 - DML also known as query language
 - 用于表达数据库的查询和更新
- Two classes of languages
 - 纯粹的 **Pure** – used for proving properties about computational power and for optimization
 - 4 Relational Algebra 关系代数
 - 4 Tuple relational calculus
 - 4 Domain relational calculus
 - 商用的(非过程化的) **Commercial** – used in commercial systems
 - 4 SQL is the most widely used commercial language



SQL

应用最广

- The most widely used commercial language
- SQL is NOT a Turing machine equivalent language
- To be able to compute complex functions SQL is usually embedded in some higher-level language
- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

不像图灵机那么强大

复杂查询要用宿主语言来写

为了访问数据库, DML 语句需要由宿主语言执行

扩展宿主语言的语法

提供应用程序接口 (应用程序: 用来与数据库进行交互的程序)



Database Design

The process of designing the general structure of the database:

- ✖ 逻辑设计(主要)
■ **Logical Design** – Deciding on the database schema.
Database design requires that we find a “good” collection of relation schemas.
- ✖ 设计(?)
● Business decision – What attributes should we record in the database?
- ✖ 理论证明
● Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **Physical Design** – Deciding on the physical layout of the database 物理设计



Database Design (Cont.)

- Is there any problem with this relation?

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Design Approaches

保证数据库关系良好

- Need to come up with a methodology to ensure that each of the relations in the database is “good”
- Two ways of doing so:
 - **Entity Relationship Model (Chapter 7)**
 - 4 Models an enterprise as a collection of *entities* and *relationships* 基本对象实体以及这些对象间的联系构成
 - 4 Represented diagrammatically by an *entity-relationship diagram*:
 - **Normalization Theory (Chapter 8)** 规范化理论
 - 4 Formalize what designs are bad, and test for them



Object-Relational Data Models

对象关系发展模型

■ **Relational model**: flat, “atomic” values

■ **Object Relational Data Models**

- Extend the relational data model by including object orientation and constructs to deal with added data types. 扩展了传统的关系模型，增加了新的特征（如结构和集合类型），以及面向对象
- Allow attributes of tuples to have complex types, including non-atomic values such as nested relations. 允许元组的属性具有复杂类型，包括非原子值，如嵌套关系。
- Preserve relational foundations, in particular the declarative access to data, while extending modeling power. 保留关系基础，特别是对数据的声明式访问，同时扩展建模能力
- Provide upward compatibility with existing relational languages. 提供与现有关系语言的向上兼容性。



XML: Extensible Markup Language

可扩展的标记语言

- Defined by the WWW Consortium ([W3C](#))
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange **data**, not just documents
- **XML has become the basis for all new generation data interchange formats.**
- A wide variety of tools is available for parsing, browsing and querying XML documents/data

XML 提供了表达含有嵌套结构的数据的方法, 能够灵活组织数据结构, 对一些非传统数据来说非常重要



Database Engine

- **Storage manager** 存储管理器 - 内存 - 磁盘.
- **Query processing** 查询处理器
- **Transaction manager** 事务管理器



Storage Management 存储管理器

存储管理器是在数据库中存储的低层数据与应用程序以及向系统提交的查询

- **Storage manager** is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system. 提供接口的程序模块。

负责的工作:

- The storage manager is responsible to the following tasks: 交互

- **Interaction with the OS file manager** 与操作系统文件管理器

- **Efficient storing, retrieving and updating of data**

- Issues: 事务 数据库中数据的存储、检索和更新

- Storage access 存储访问

- File organization 文件组织

- Indexing and hashing 索引和哈希

↓
B⁺树(平衡树)



Storage Hierarchy

高速缓冲存储器：
与数据库无关

cache

main memory

flash memory

快闪存储器：
USB口存取，
与数据库无关

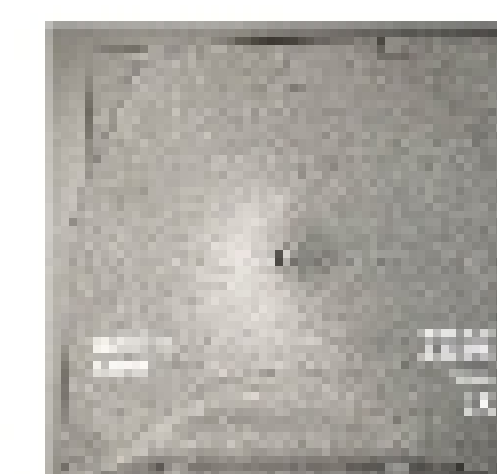
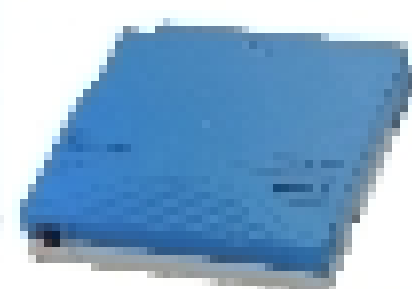
magnetic disk 磁盘

optical disk 光盘

magnetic tapes 磁带

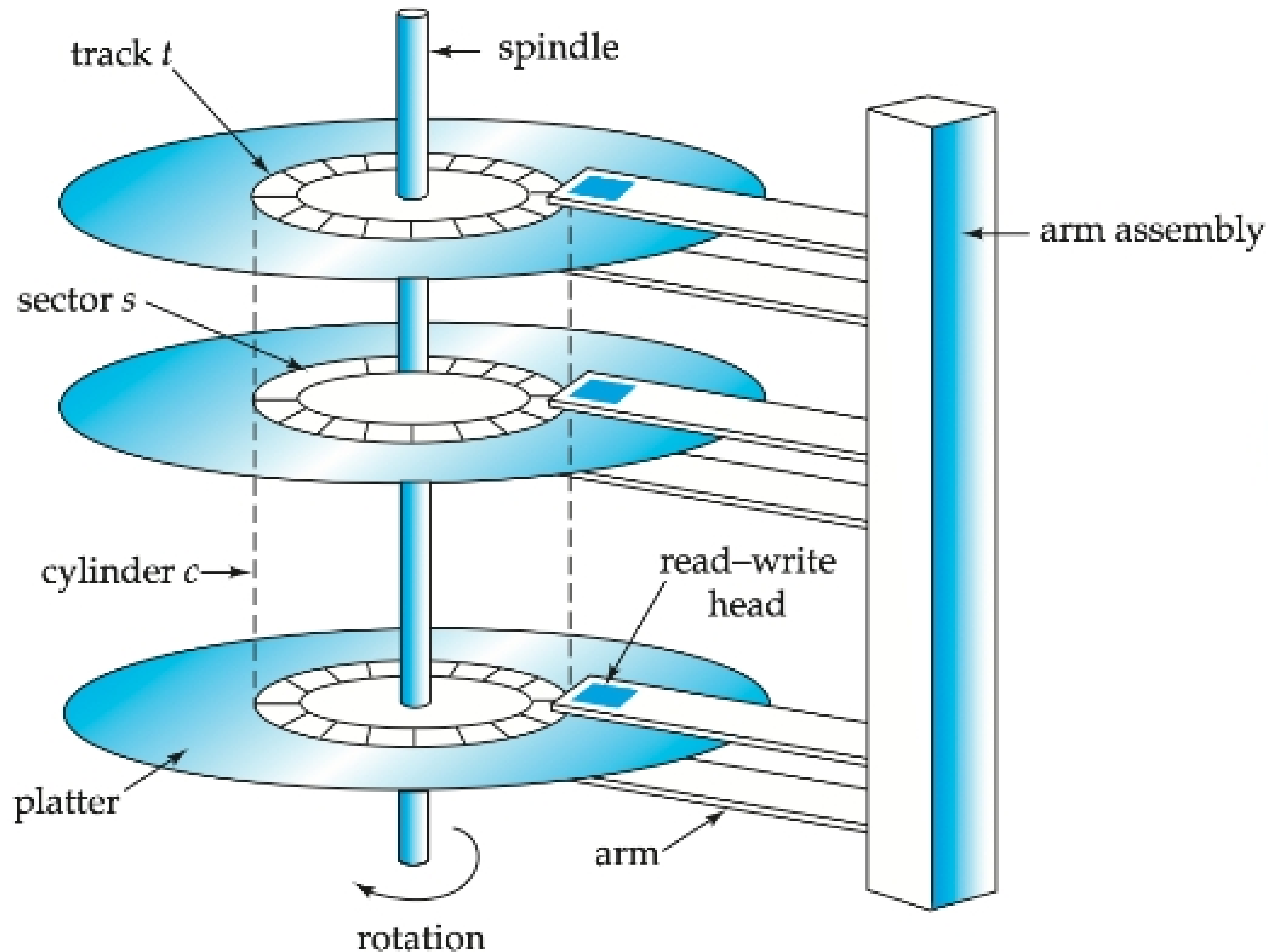


HPE LTO-5 Ultrium 3TB



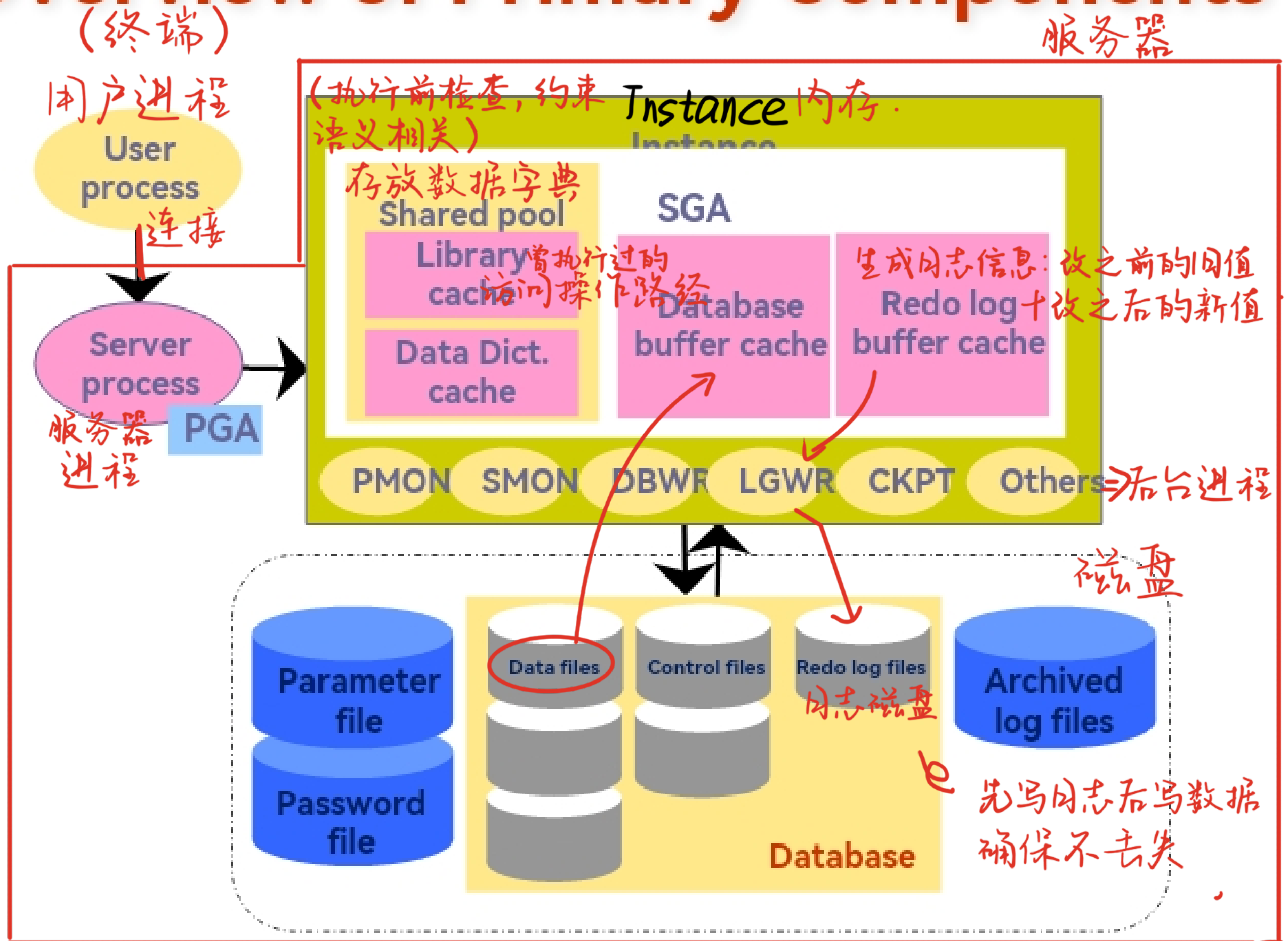


Magnetic Hard Disk Mechanism





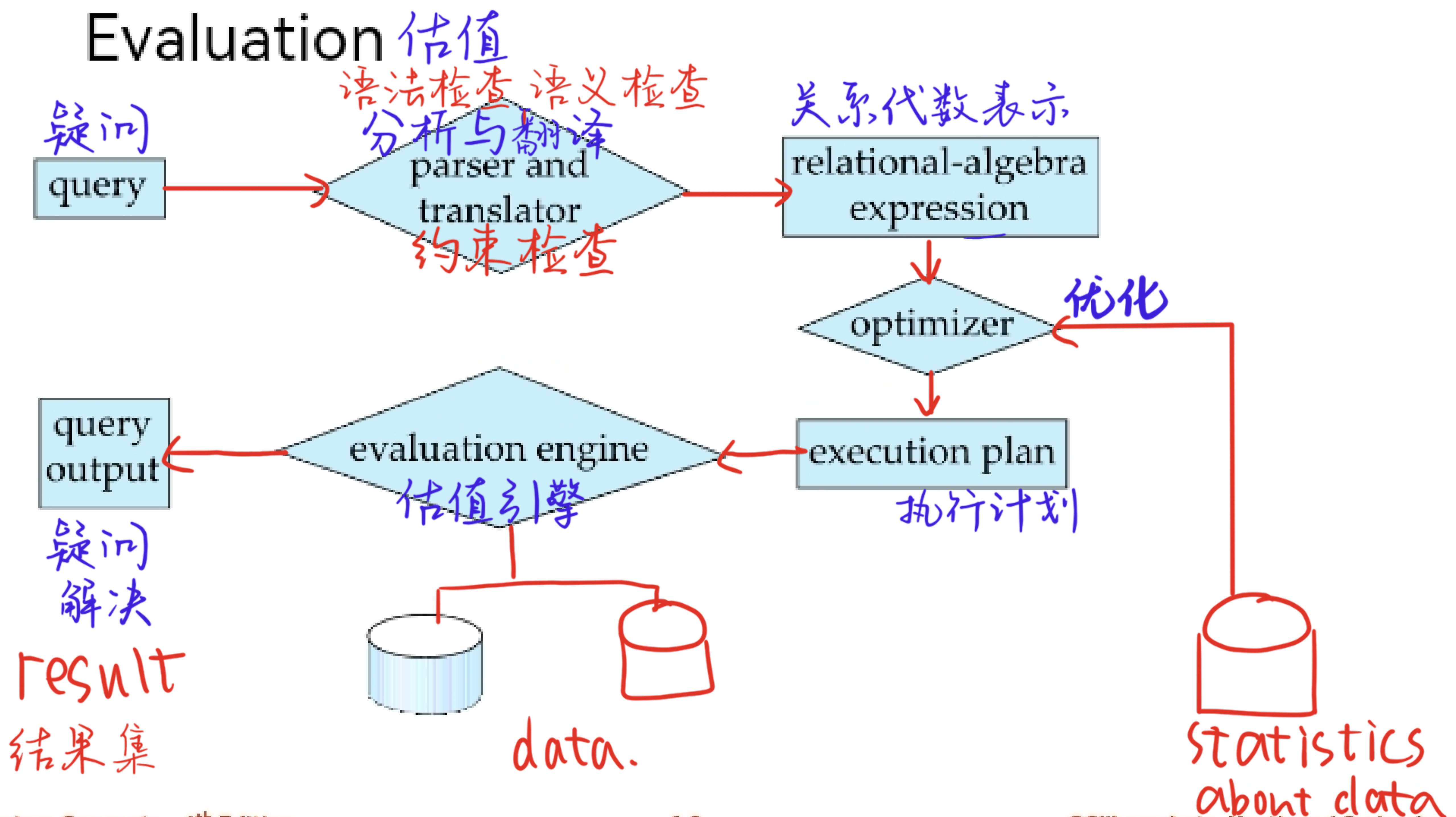
Overview of Primary Components





Query Processing 查询处理

1. Parsing and translation 解析和翻译
2. Optimization 最优化
3. Evaluation 估值





Query Processing (Cont.)

疑问处理(续)

评估一个给出的疑问的可替换方法

- Alternative ways of evaluating a given query
 - Equivalent expressions 等效公式替换
 - Different algorithms for each operation 每一步的不同算法
- Cost difference between a good and a bad way of evaluating a query can be enormous 评估算法好坏的不同是很费力的
- Need to estimate the cost of operations 需要估计操作的开销
 - Depends critically on statistical information about relations which the database must maintain 严格按照数据库必需保养的数据信息关系
 - Need to estimate statistics for intermediate results to compute cost of complex expressions 需要评估统计中间结果以估算复杂公式的开销



Transaction Management 事务管理

- What if the system fails? 如果系统瘫痪怎么办?
- What if more than one user is concurrently updating the same data? 如果不只一个用户同时上传相同的数据怎么办?
- A **transaction** is a collection of operations that performs a single logical function in a database application. 事物是指在数据库应用中单一的、逻辑的操作集合 既有原子性 又有一致性
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures. 事物管理部件确保了即使系统瘫痪数据库仍可以恢复到该失败事务开始执行前的状态
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database. 并发控制管理器控制并发事务间的相互影响, 保证数据库一致性



Database Users and Administrators

数据库用户和管理员

(出纳员, 代理, 网络用户)

初级用户

naive users
(tellers, agents,
web users)

use

应用界面
application
interfaces

应用程序员

application
programmers

write

应用程序
application
programs

(分析员)

富有经验的用户

sophisticated
users
(analysts)

use

查询工具
query
tools

数据库管理员

database
administrators

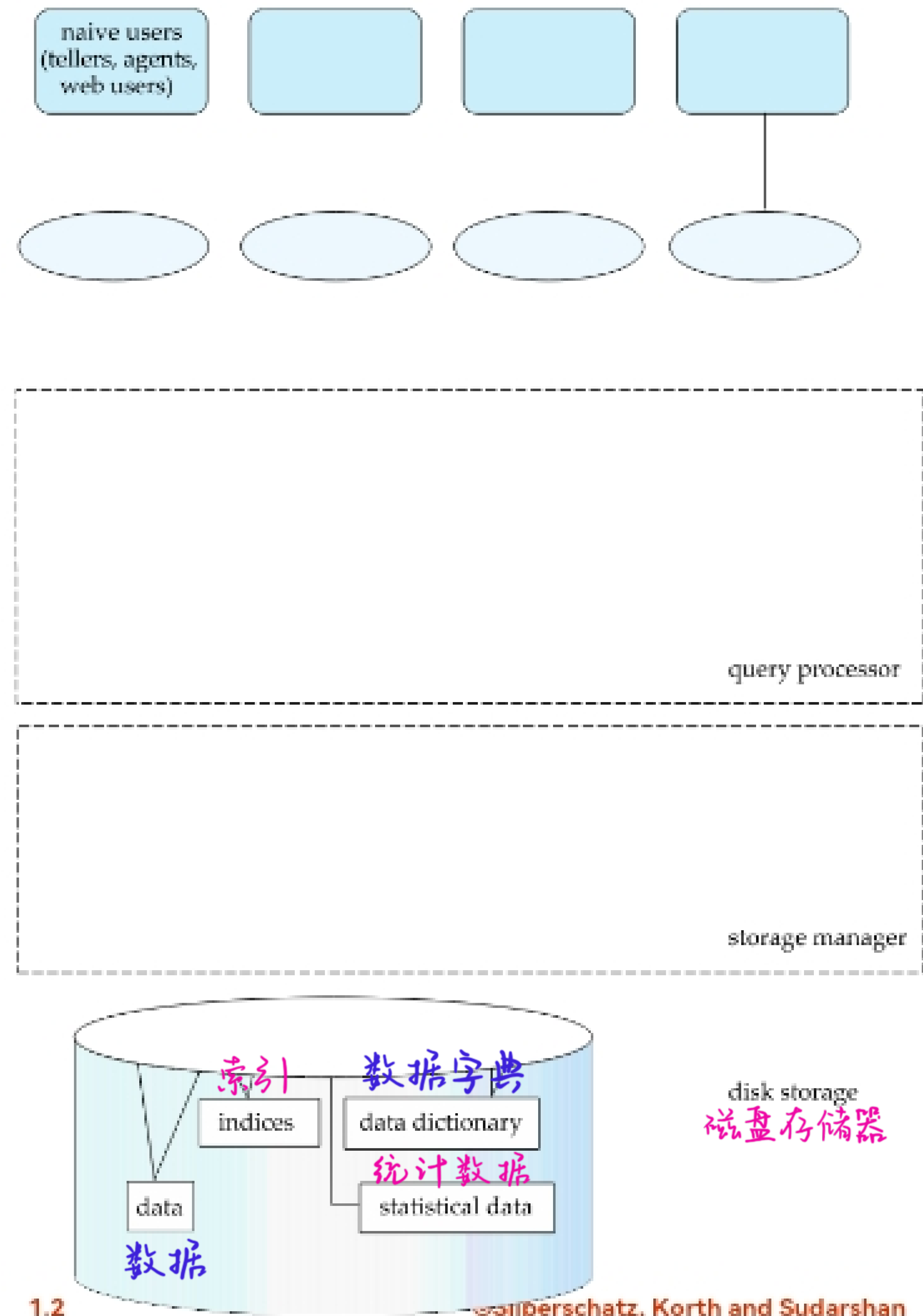
use

管理工具
administration
tools

Database



Database System Internals





Database Architecture 数据库结构

数据库结构的名称

The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

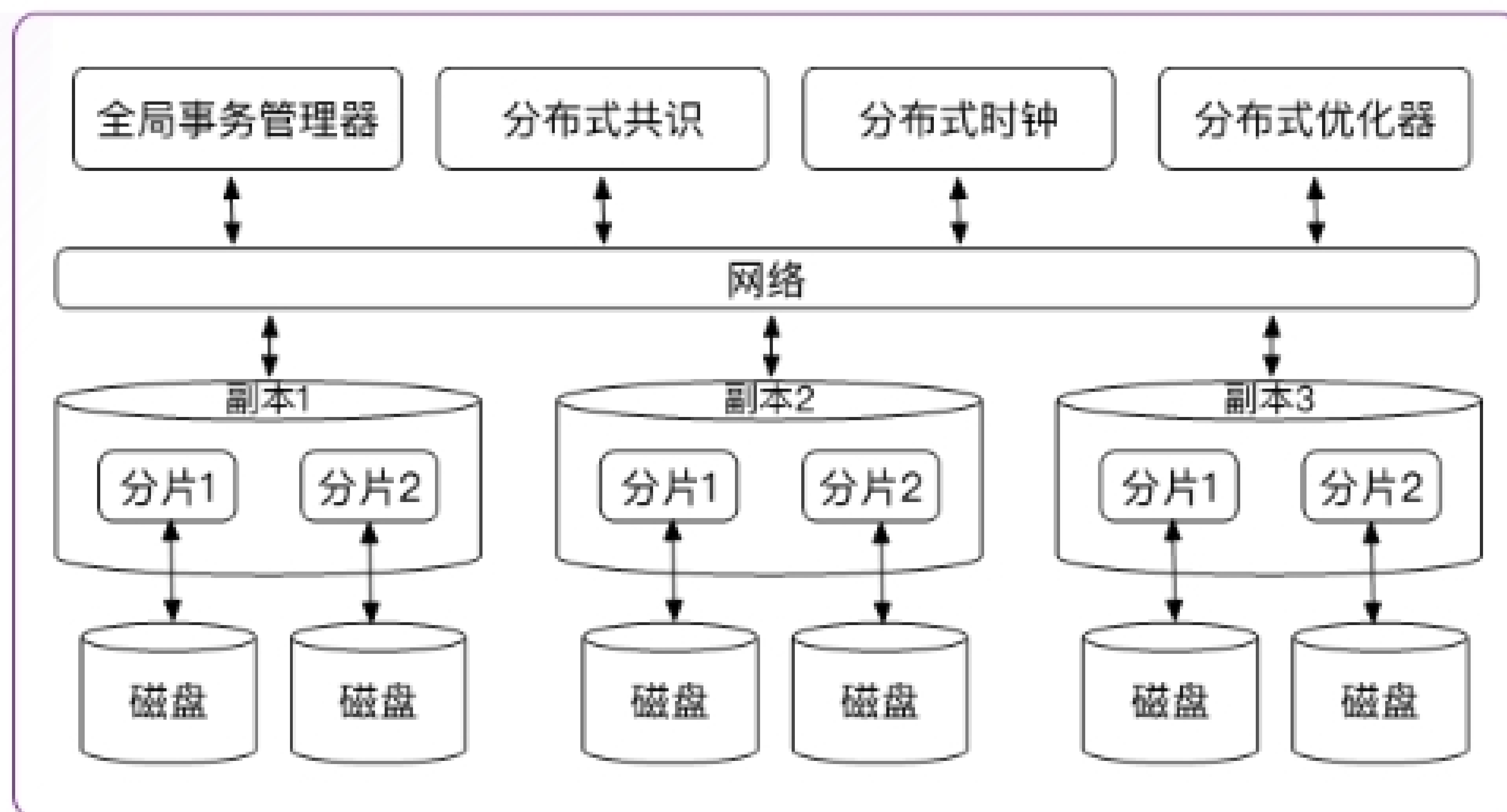
- Centralized 集中结构
 - Client-server 客户端-服务器架构
 - Parallel (multi-processor) 并行(多处理器)系统
 - Distributed 分布式系统
- 系统内部 普通用户
↑ ↑
C/S + B/S 常用: 客户端-服务器
+ 网络-服务器



数据库架构：分布式数据库

产生背景

- 海量数据存储
 - TB、PB级数据
- 高可用应用需求
 - 电商、通信、金融应用
- 分布式数据库
 - 高性能
 - 高可用
 - 高可扩展





数据库架构：云托管数据库

□ 云托管数据库系统特点：

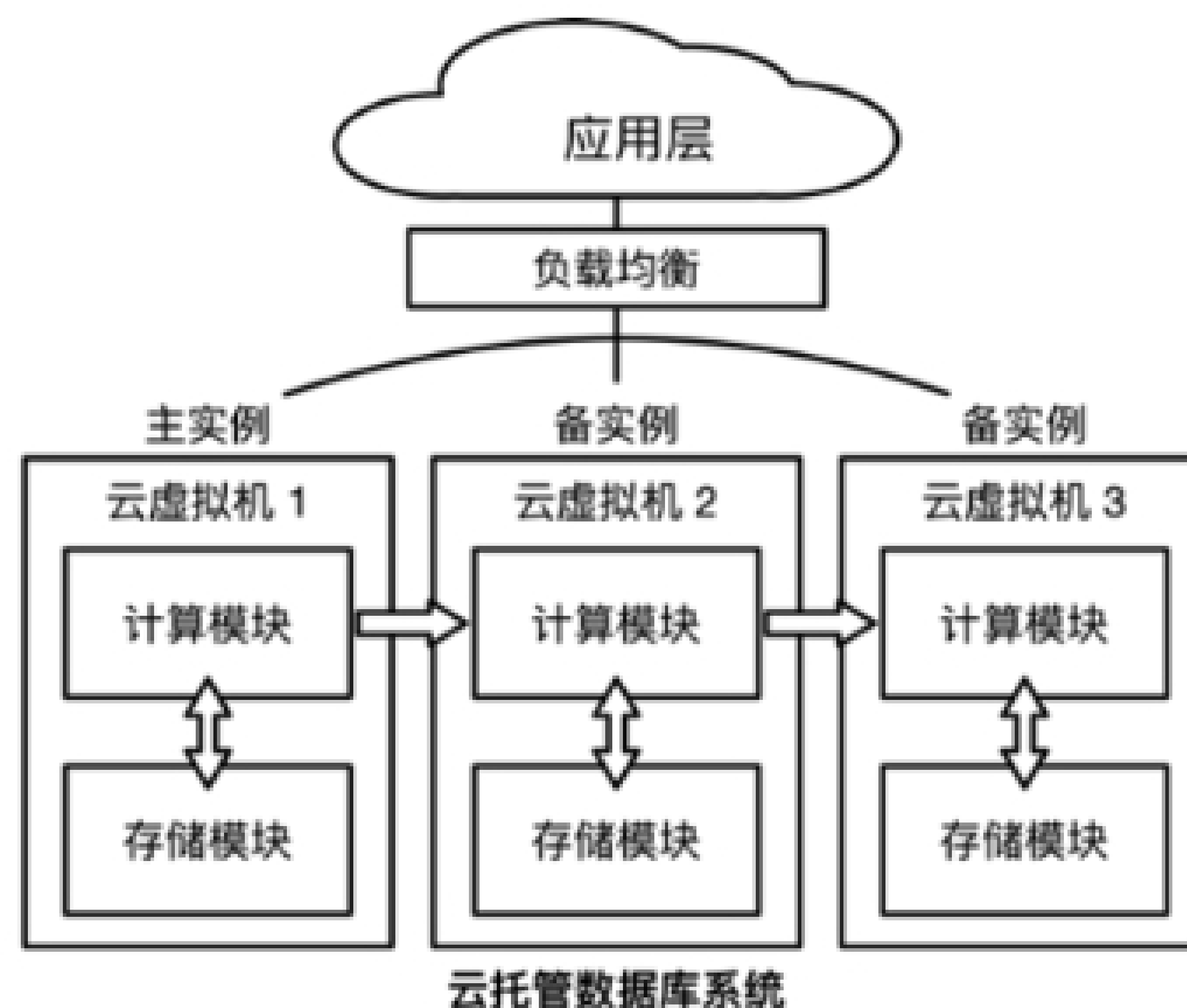
- 多个数据库实例
- 总体为Shared Everything结构

□ 优势：

- 易于实现，完全兼容

□ 缺陷：

- 本机存储容量
- 冗余的数据副本
- 高RTO，数据滞后
- 网络瓶颈





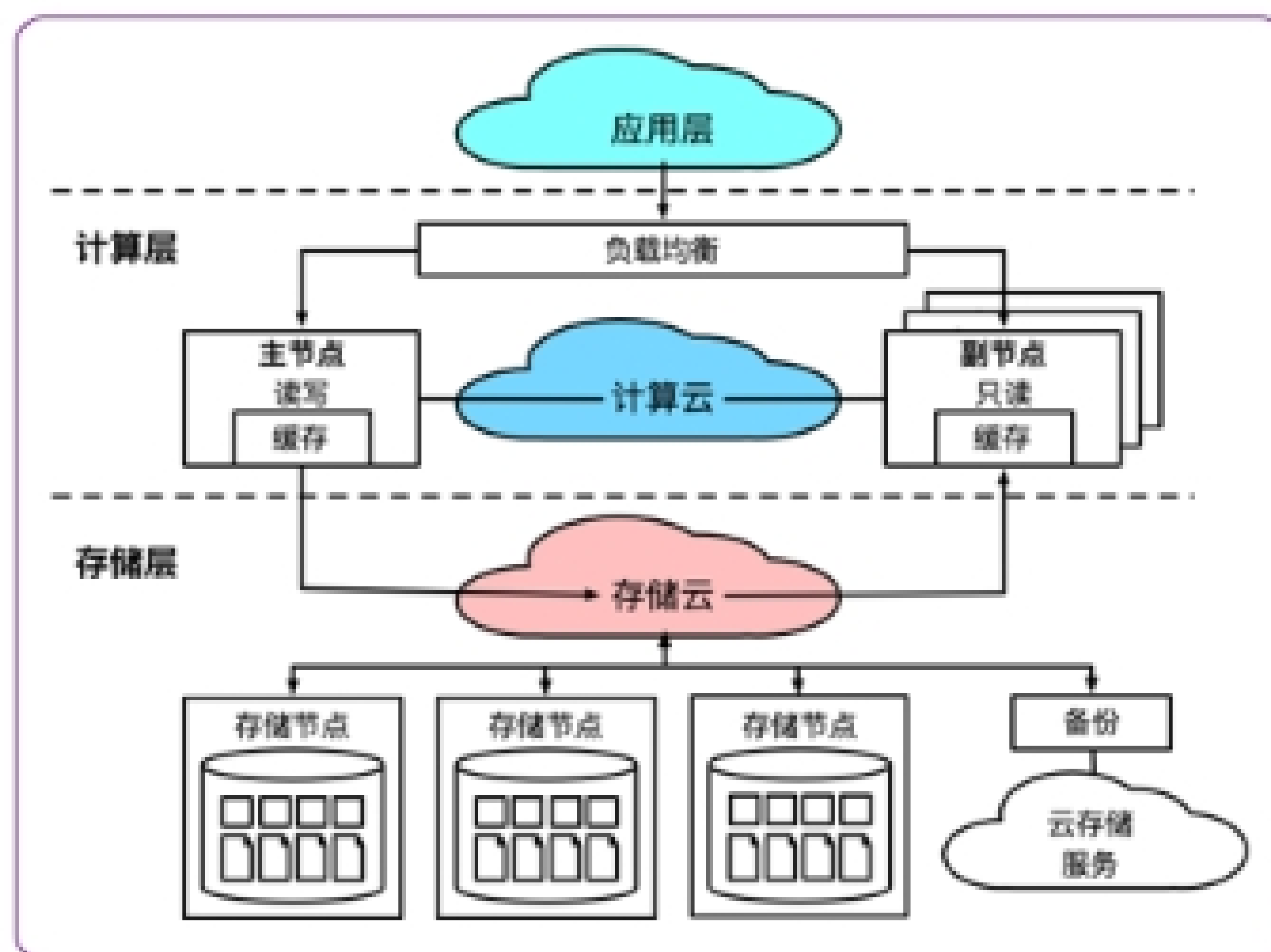
数据库架构：云原生数据库

□ 计算-存储分离架构优势:

- **弹性** 计算资源和存储资源可以独立调度。
- **效率** Shared Storage架构，缓解写放大问题。
- **可用性** 节点级别的故障恢复，降低故障恢复时间。

□ 关键特性:

- 计算-存储分离
- 日志即数据库





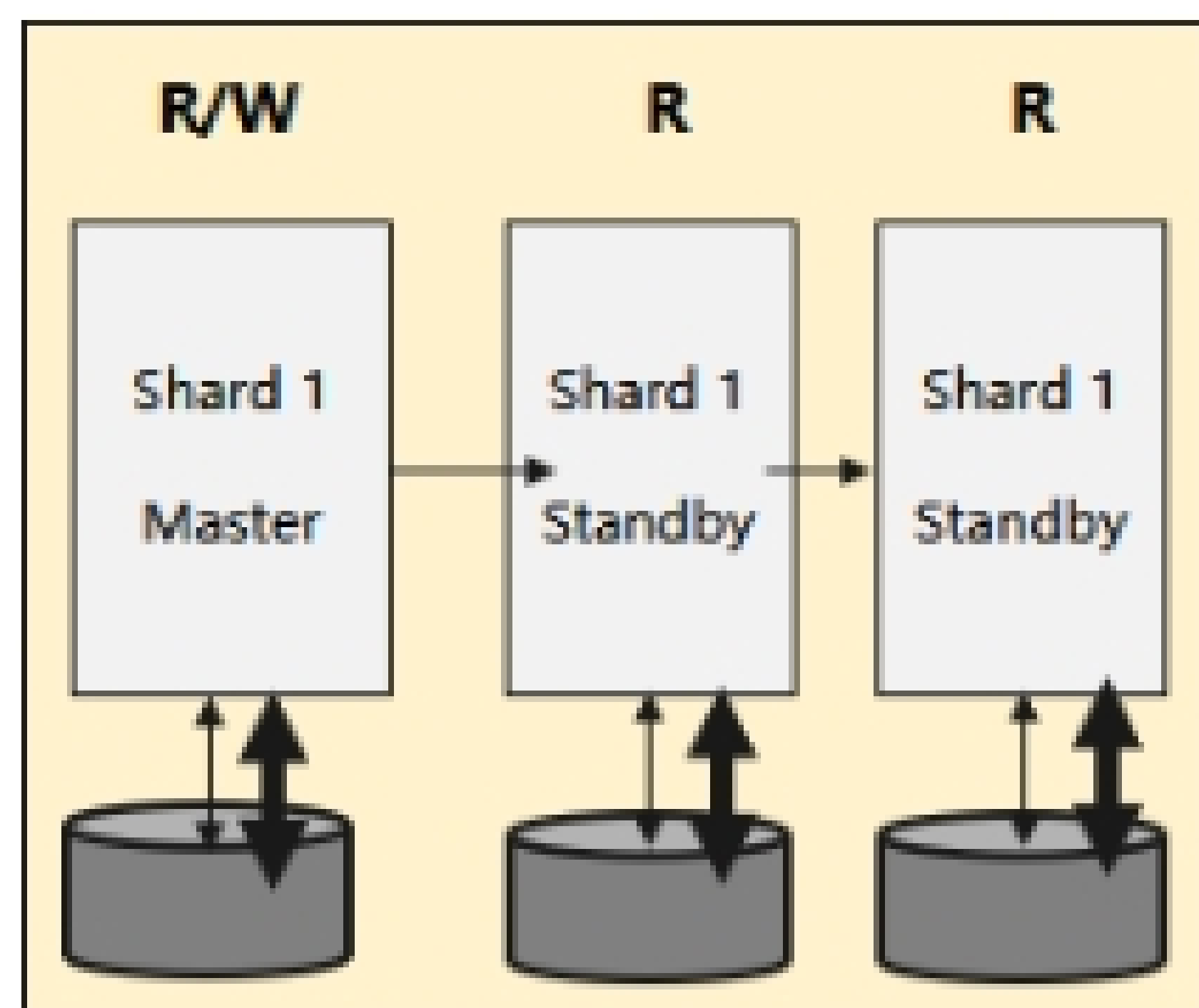
数据库架构：云原生数据库

□ 核心区别：架构设计上的计算存储分离

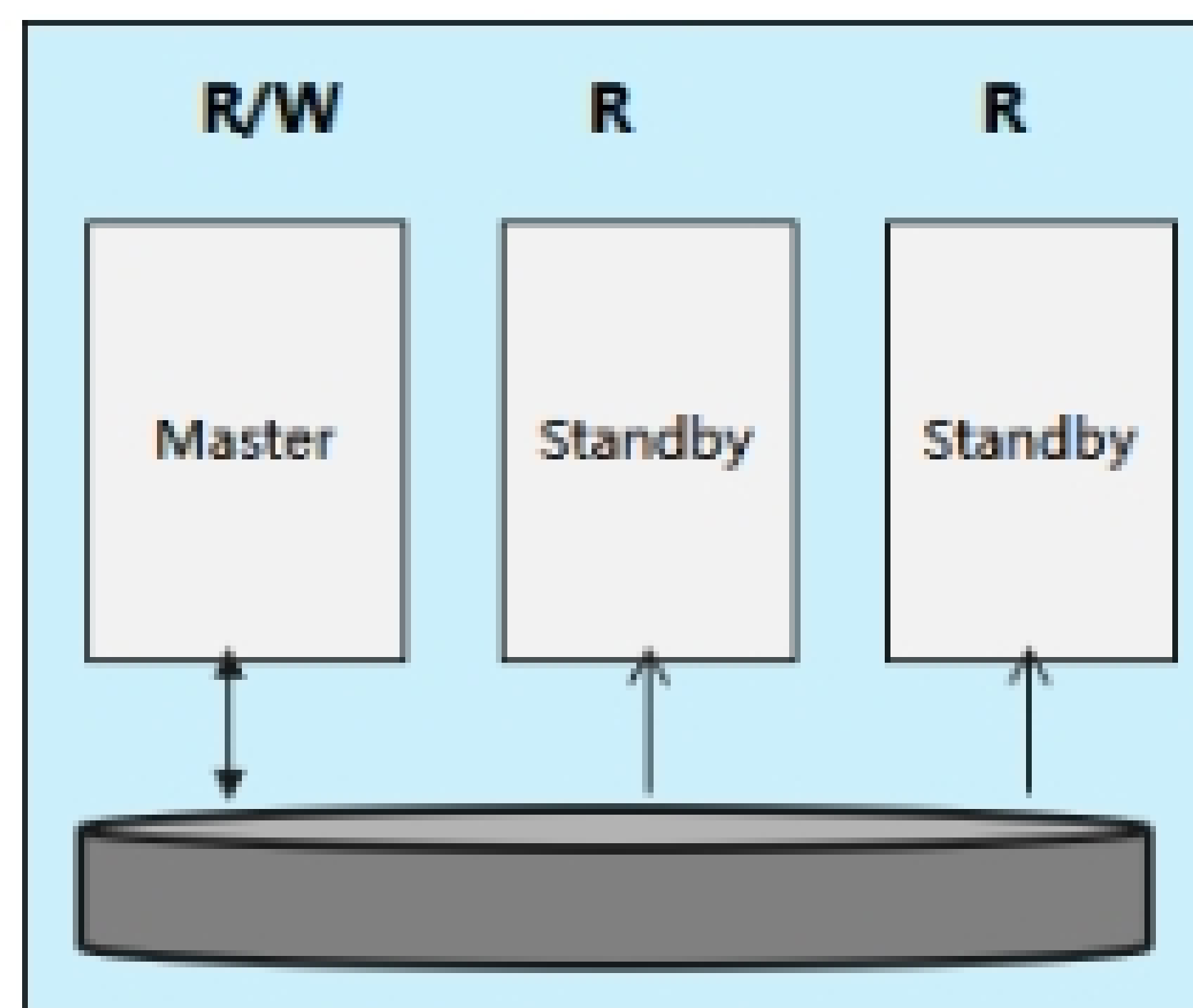
- OLTP:注重事务处理，日志为核心
- OLAP:强调数据分析，分片为核心

□ 关键技术：

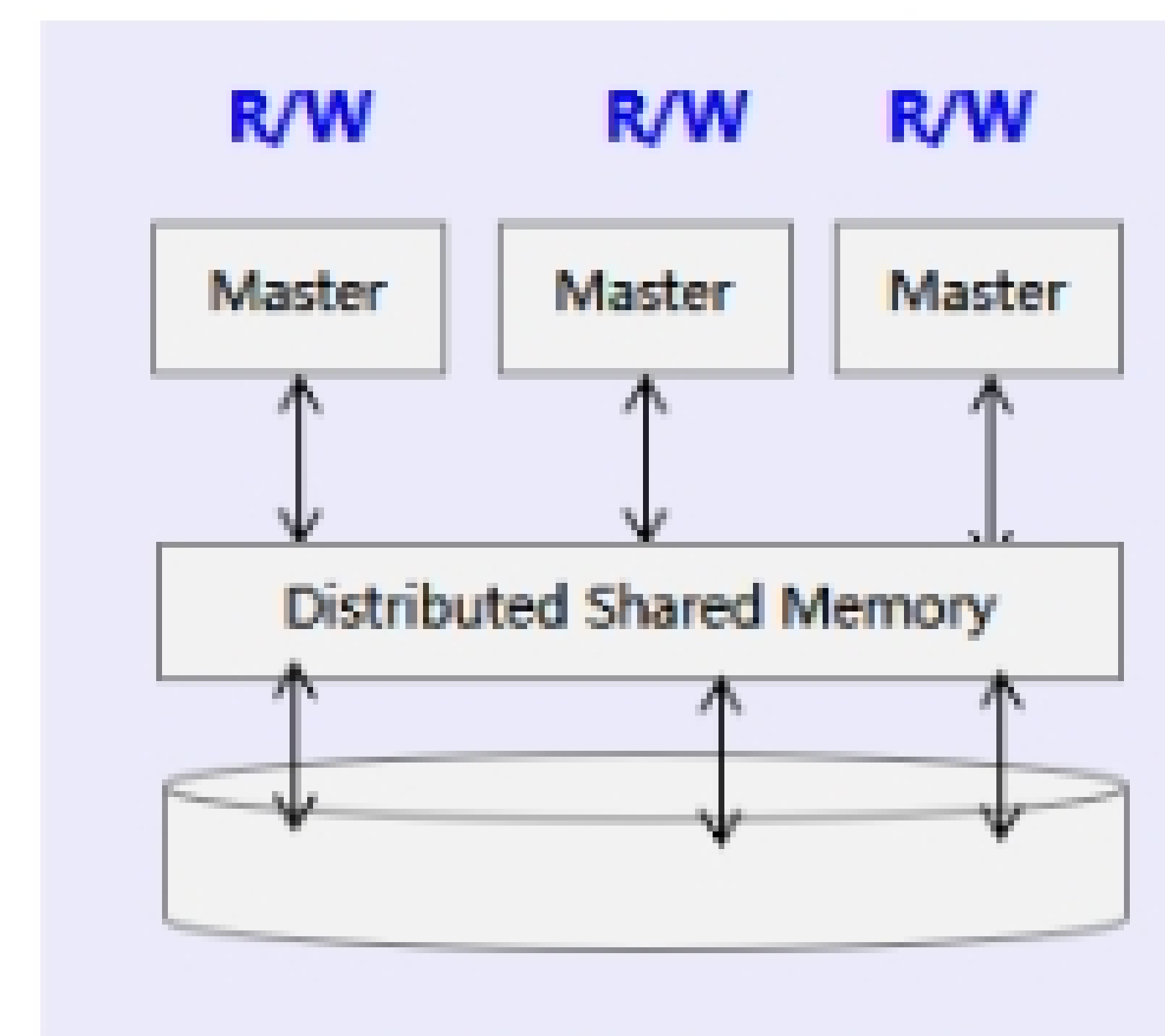
- 事务处理
- 数据副本
- 异步更新
- 故障恢复
- 分析处理



传统数据库



CloudDB1.0:
计算存储分离



Cloud DB 2.0:
三层池化、多写多读



History of Database Systems

数据库系统历史。

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - 4 Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model
 - 4 Would win the ACM Turing Award for this work
 - 4 IBM Research begins System R prototype
 - 4 UC Berkeley begins Ingres prototype
 - High-performance (for the era) transaction processing



History (cont.)

- 1980s:
 - Research relational prototypes evolve into commercial systems
 - 4 SQL becomes industrial standard
 - Parallel and distributed database systems
 - Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce
- Early 2000s:
 - XML and XQuery standards
 - Automated database administration
- Later 2000s:
 - Giant data storage systems
 - 4 Google BigTable, Yahoo PNuts, Amazon, ..



数据库模型： NoSQL数据库类型

□ 键值数据库

- 以键值对格式存储数据的数据库，数据访问简单高效

□ 列族数据库

- 数据列按访问关联性划分成不同族，一个族可视为关系表

□ 文档数据库

- 以JSON等文档格式存取数据，避免应用中数据转换的代价

□ 图数据库

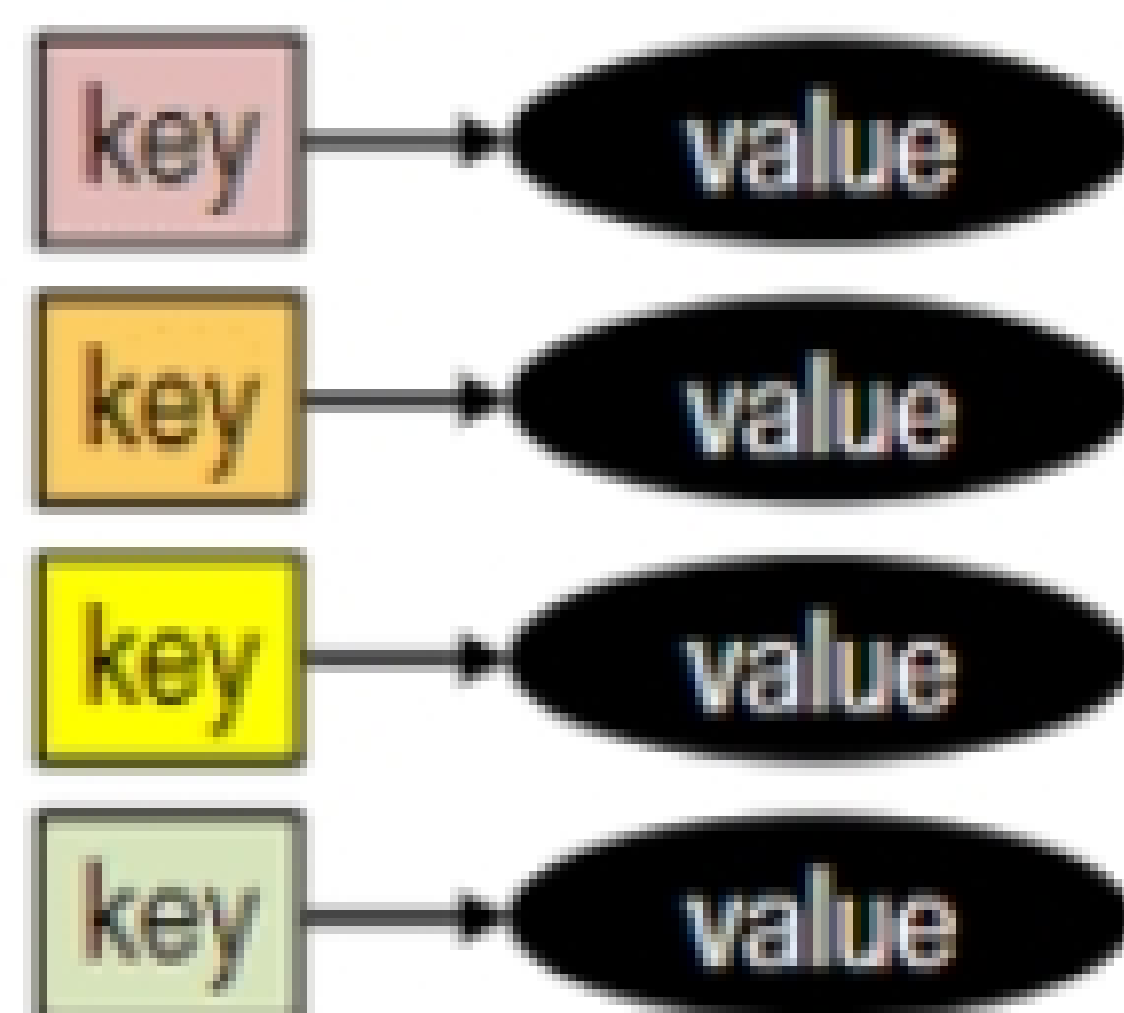
- 以包含边、节点及属性的图结构来存储数据和建模关联信息

□ 向量数据库

- 非结构化数据（文本、图片）转换的向量数据（embedding、vector）

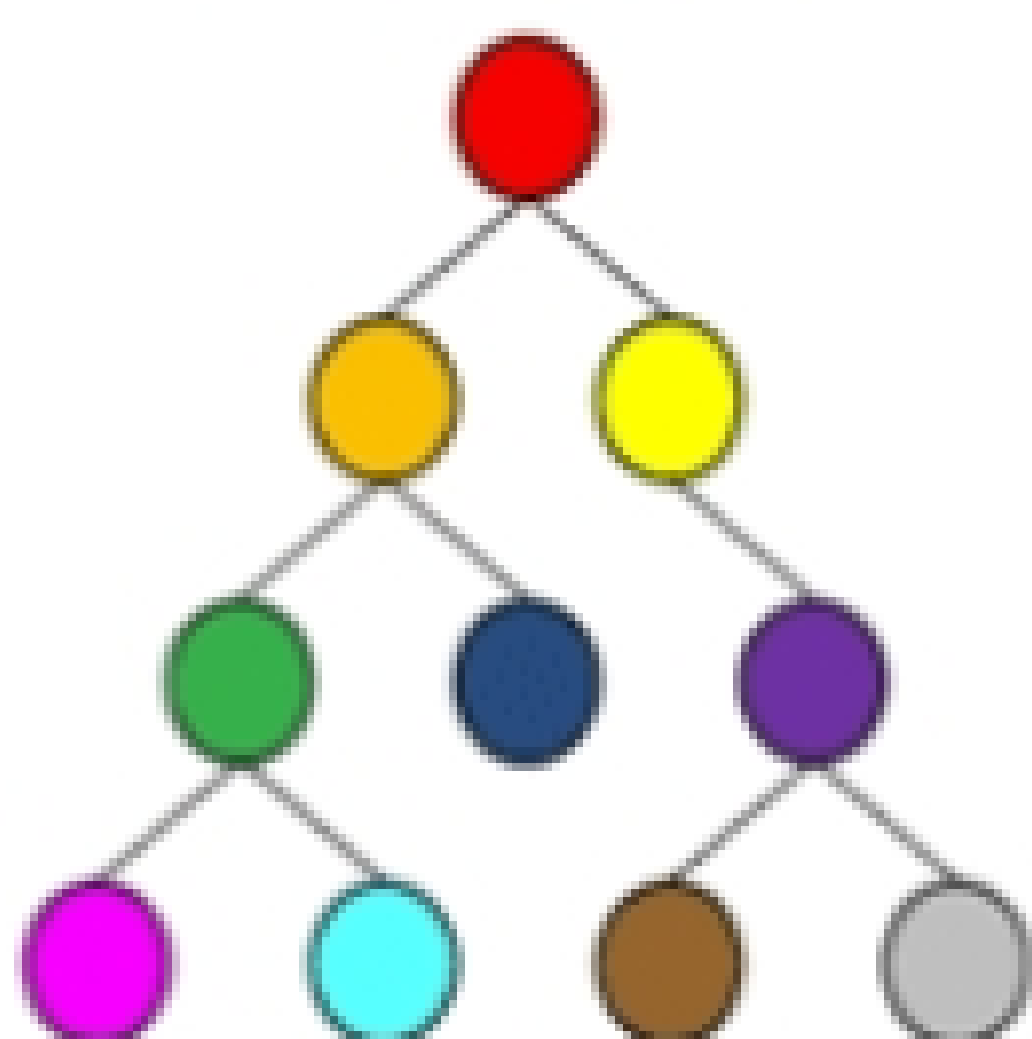


列值数据库 Key-Value



Row Key-1	Column-Family-1		Column-Family-2	
	Column Name-1	Column Name-2	Column Name-3	
	Column Value-1	Column Value-2	Column Value-3	
Row Key-2	Column-Family-1			
	Column Name-4	Column Name-5	Column Name-6	
	Column Value-4	Column Value-5	Column Value-6	

Document



Graph



vector

0.03, 0.12, 0.01,
0.01, 0.01, 0.02,
0.02, 0.02, 0.03,
0.04, 0.11, 0.05,
⋮
0.11, 0.13, 0.01,
0.06, 0.01, 0.02,
0.02, 0.02, 0.03,



数据库模型： NoSQL数据库主要特点

NoSQL = Not only SQL

模式灵活

NoSQL数据库无需预先指定数据的模式，并支持混合数据类型

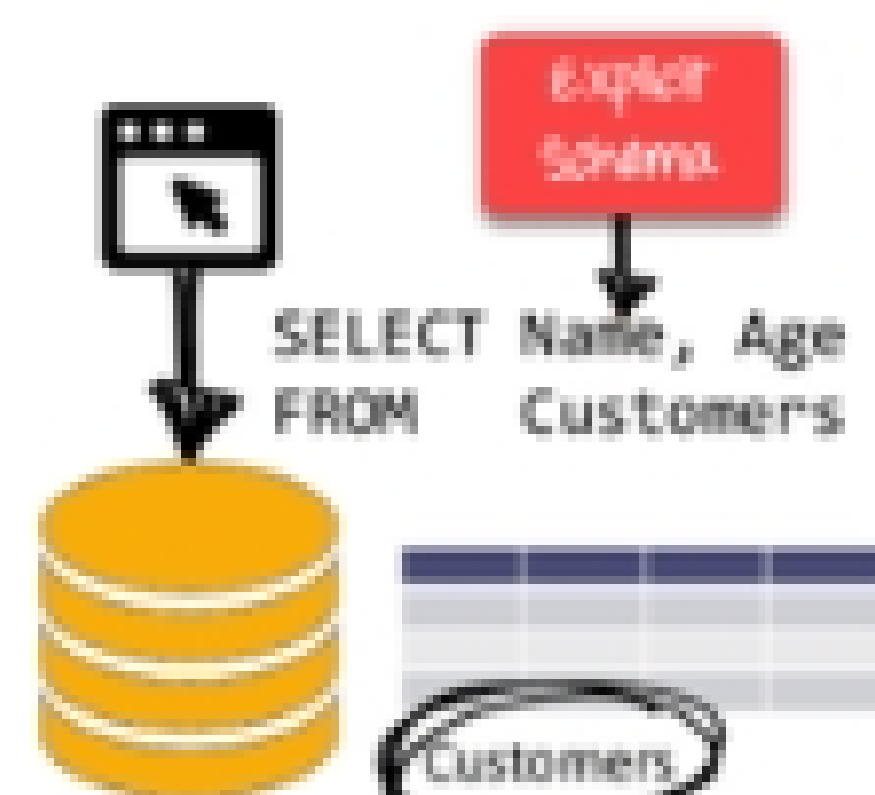
高可扩展

摆脱关系模型约束，NoSQL简化提升了数据水平扩展能力

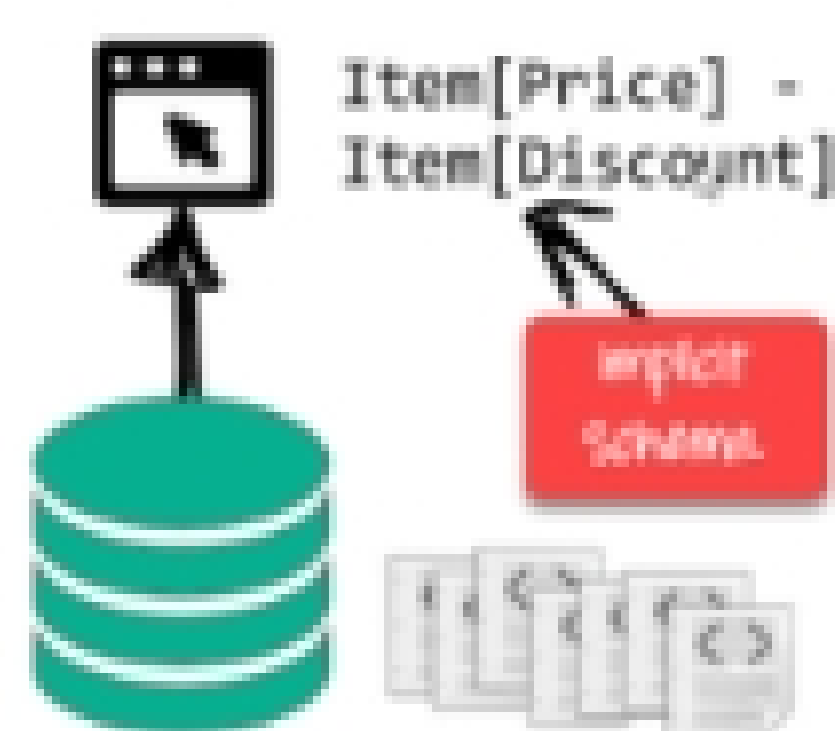
高可用

基于主主或主从集群模式，支持用户从不同数据节点查询数据

RDBMS:



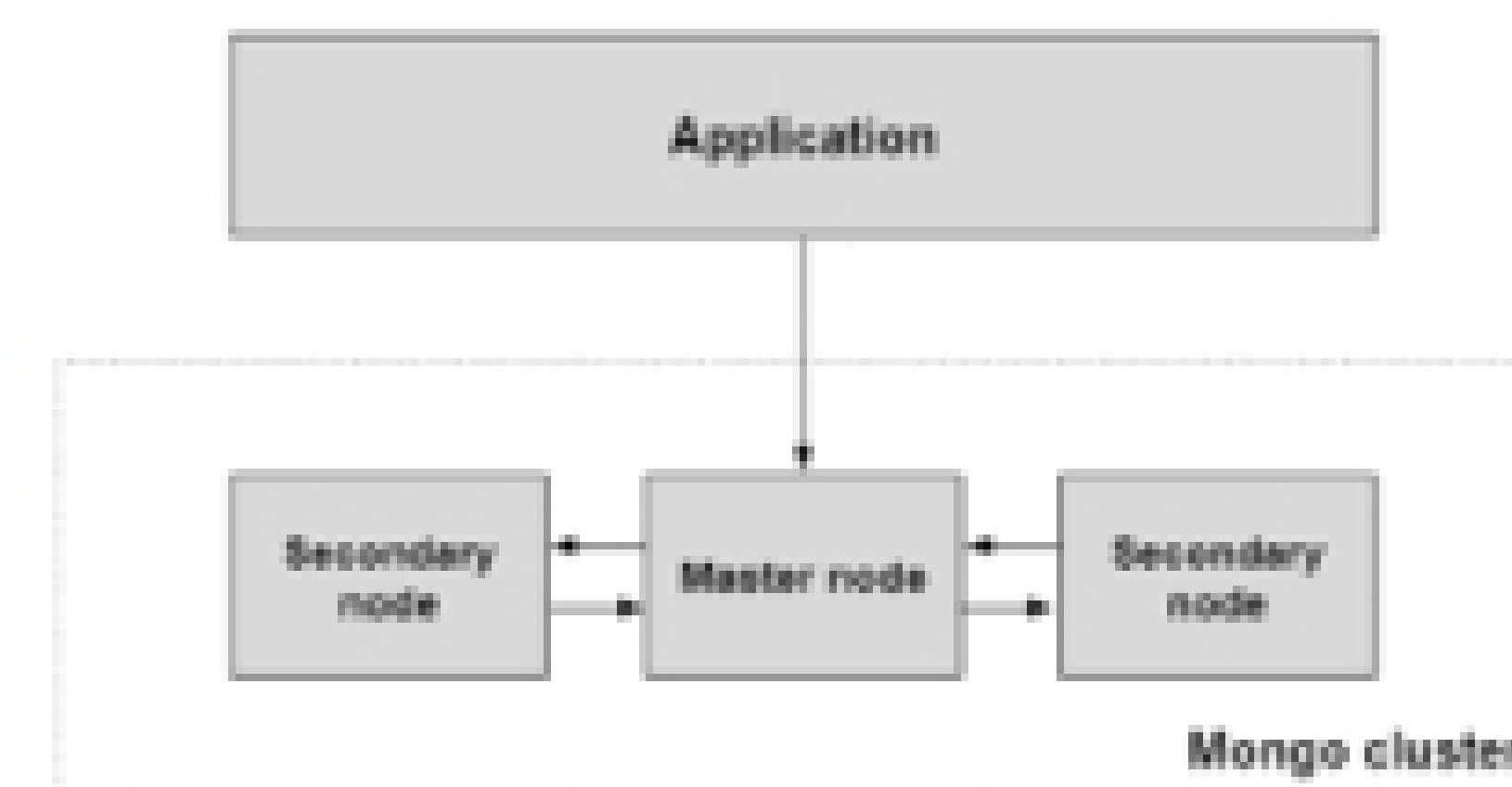
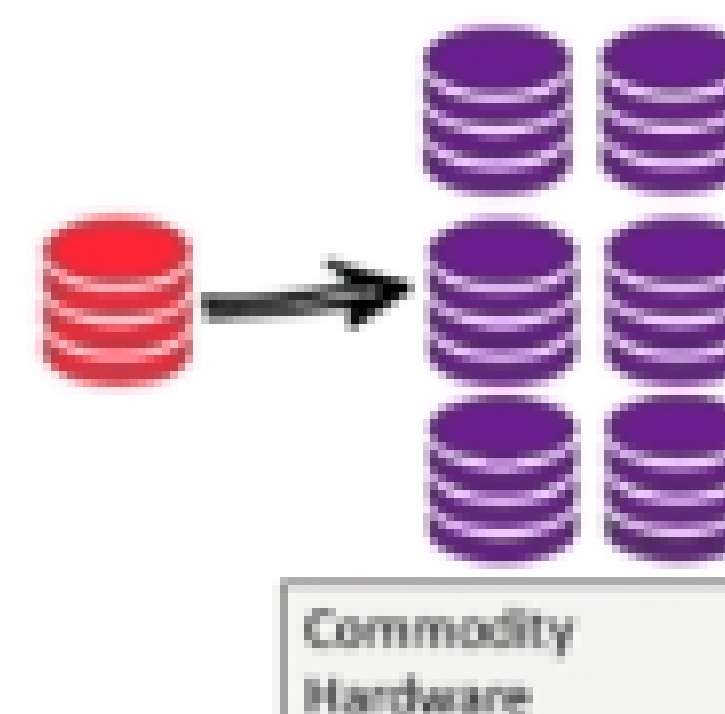
NoSQL DB:



Scale-Up (vertical scaling):



Scale-Out (horizontal scaling):





数据库模型：常见的NoSQL数据库

主要类型		NoSQL数据库产品（示例）
键值数据库	内存键值存储	Redis, Memcached
	最终一致性键值存储	Dynamo
	持久化键值存储	LevelDB、RocksDB
列族数据库		Cassandra, Hbase, Google BigTable
文档数据库		MongoDB
图数据库		Neo4J
向量数据库		Milvus



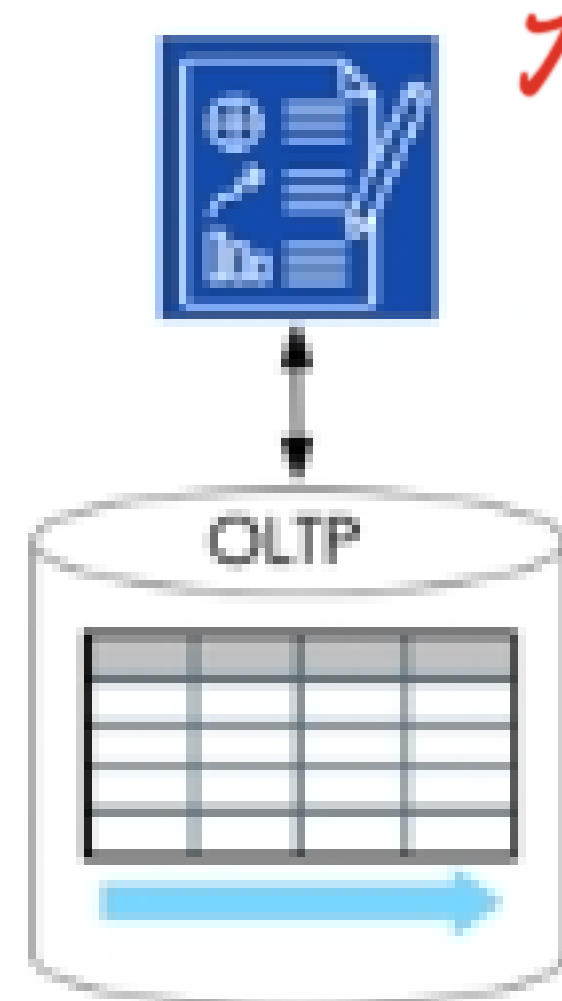
数据库各类应用：OLTP、OLAP与HTAP数据库

(联机事务处理)

- OLTP: Online Transaction Processing 在线事务处理
- OLAP: Online Analytical Processing 在线分析处理 (联机分析处理)
- HTAP: Hybrid Transactional/Analytical Processing 混合事务分析处理

OLTP

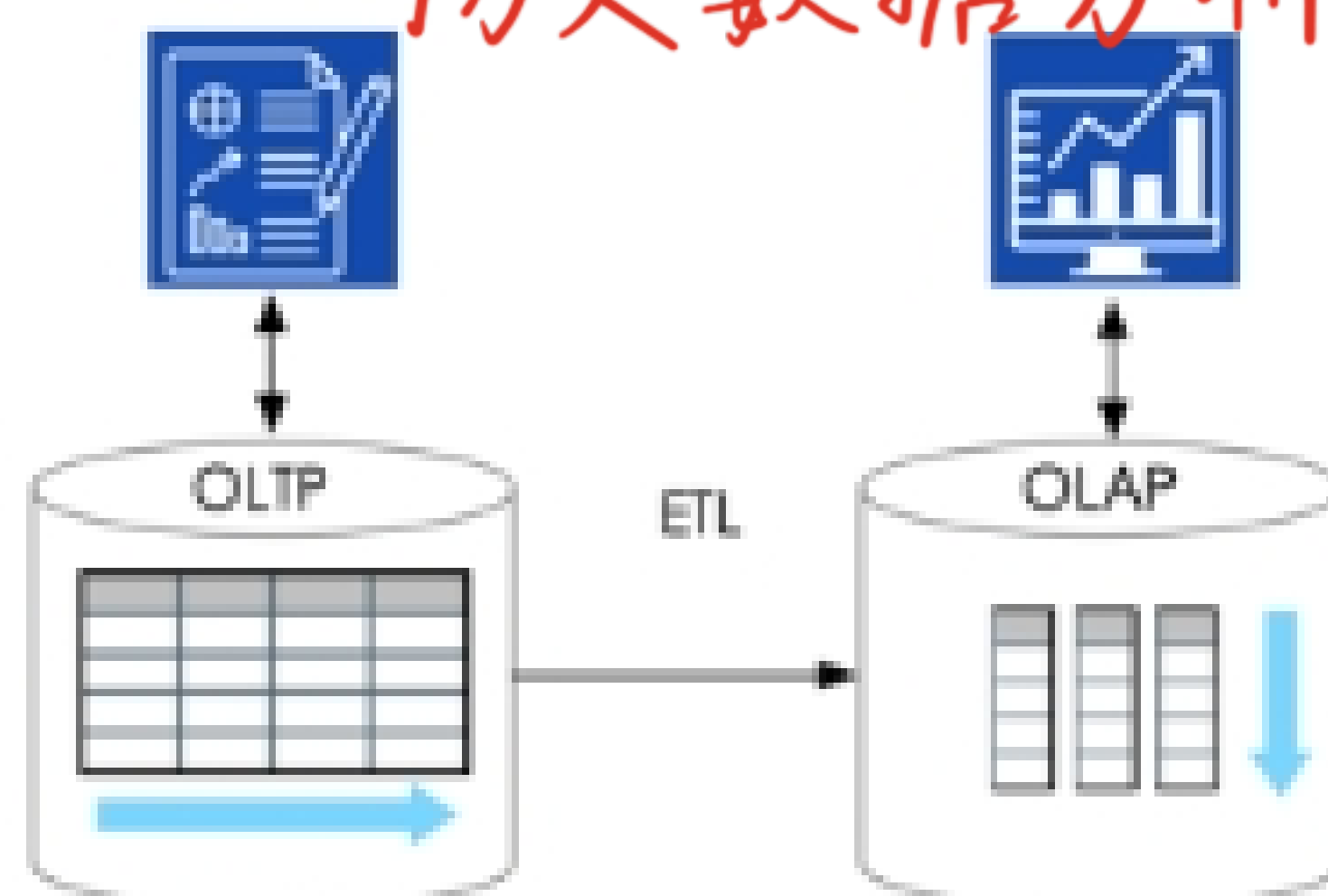
大量写操作



- 动机：处理业务交易数据
- OLTP每一条写入操作往往对应一条业务记录。

OLAP

历史数据分析与预测

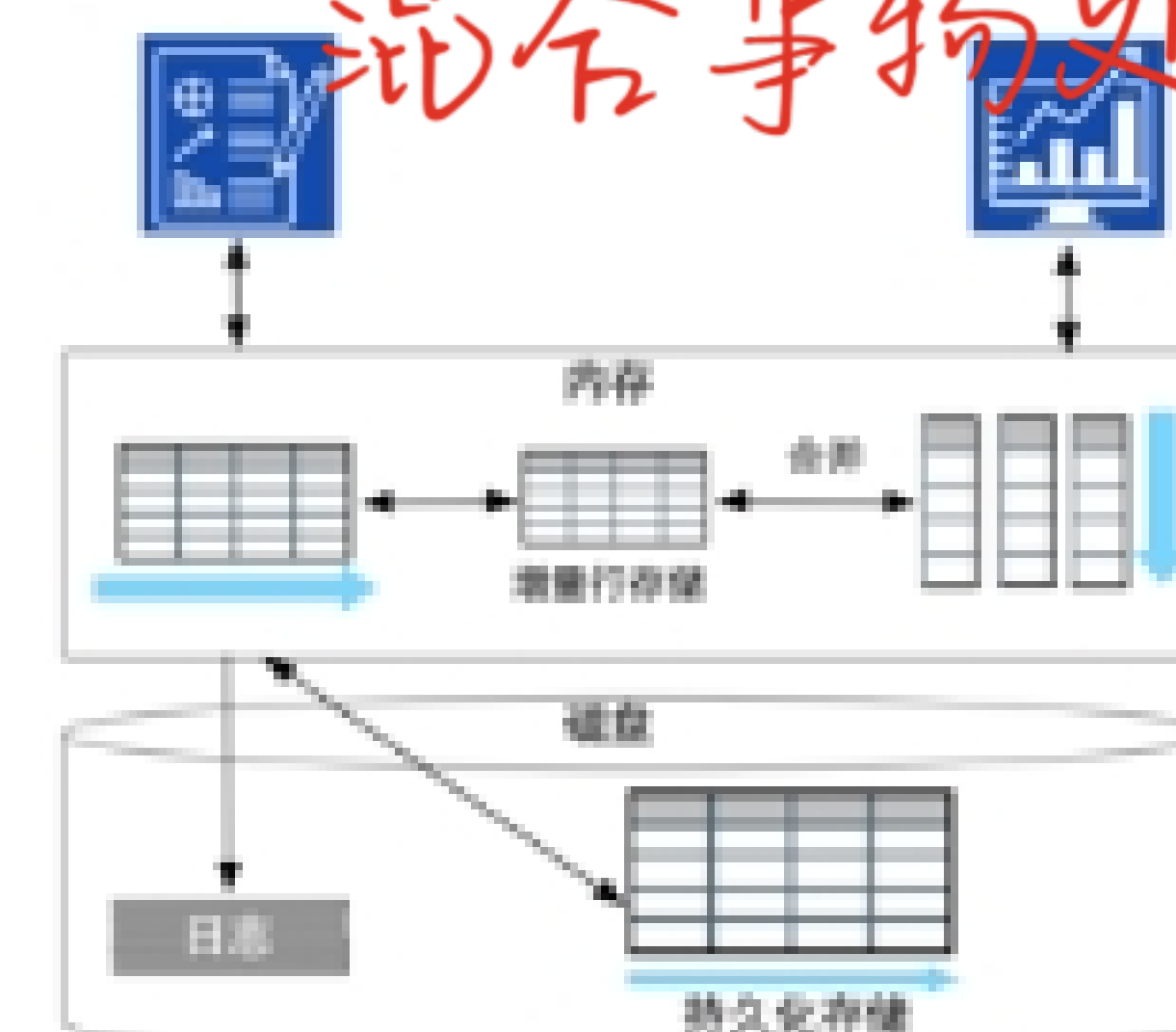


- 动机：从日积月累的大量数据中做数据分析，挖掘数据的价值
- OLAP定期从OLTP经过ETL提取数据，用于数据分析。

HTAP

混合事物处理

分析+预测



- 动机：需要对最新事务信息作出快速的分析，甚至需要在处理事务的同时对数据进行分析。
- HTAP是能够同时处理并发型事务与分析型查询的数据库技术。

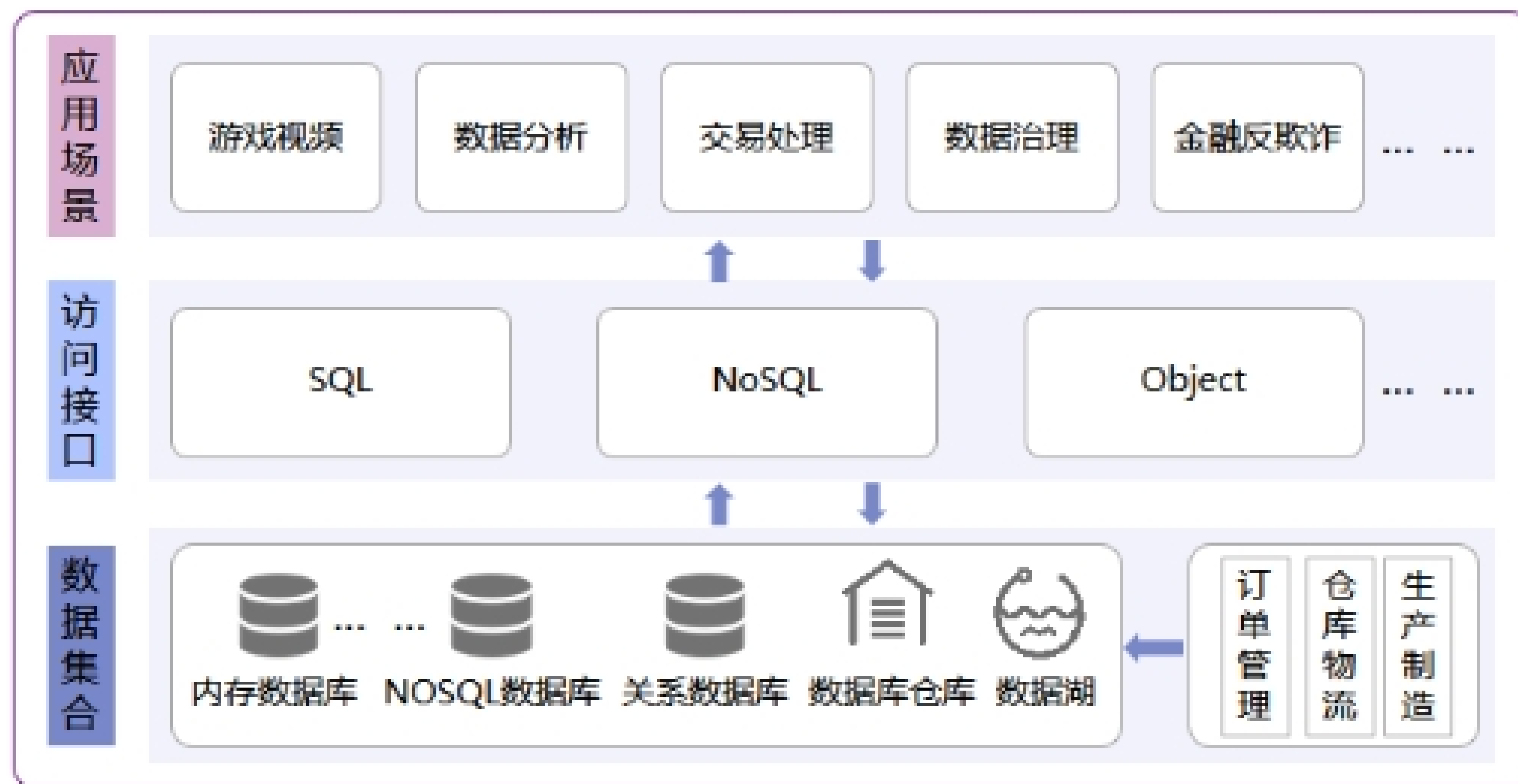


数据库各类应用

管理信息系统

事务处理系统

决策支持系统





2020云上双11

阿里云扛住全球最大流量洪峰







End of Chapter 1